

Data-driven Modeling of a Multi-coil Magnetic Actuation System

Gediyon Moges Girma

Supervisors: Dr. Eric Diller, Daniel Sieben, Erik Fredin

December 2024

Abstract

Previously, system characterization of an 8-coil magnetic actuation system was performed by measuring the magnetic field generated for input currents ranging from -13 to 13 Amps, in 2 Amp increments per coil. This was aimed at expanding magnetic actuation control from a single calibrated point to a 3D workspace. Building on this foundation, the current project focuses on developing a data-driven model to predict magnetic field values in a 3D workspace based on input current. To ensure high-quality data, additional direct measurements were collected for randomized input values. Using this dataset, model optimization was carried out, resulting in an optimized model with a mean absolute percentage error (MAPE) of approximately 4% during testing exceeding the accuracy of models reviewed in the literature. However, test case evaluations indicated that the model could benefit from further training with additional randomized input values. Furthermore, this study uncovered an unexpected lack of correlation between the command and measured current values, indicating the need for further investigation to identify the root cause of this issue.

Contents

1	Introduction	2
2	Background	2
3	Literature Review	2
3.1	Machine learning models and Artificial neural networks (ANNs)	2
3.2	Embedding Physics into Neural Networks	4
3.3	Previous Studies on Electromagnetic Field Prediction	5
4	Objectives	5
4.1	Specific Objective	5
5	Approach	5
6	Result	6
6.1	Model architecture Tuning	6
6.2	Hyper-parameter tuning	7
6.3	Optimized model	8
6.4	Model prediction vs Measurement	8
6.5	Performance comparison between FFN model, dipole model and susperpositioned data . .	9
6.6	Measured vs command current	10
7	Conclusion and Recommendation	10
A	GUI for Randomized Dataset Collection	13
B	Training Data Distribution In The Workspace	14
C	Input Current Vector Distribution of the Training Dataset	15
D	FFN Prediction vs Measured Data	16
E	Command vs Measured Current of the Training Dataset	17

1 Introduction

An 8-coil magnetic actuation system[11], developed in the Micro-robotics lab at the University of Toronto, has been empirically characterized in the first part of this project [2]. Initially, the system's response was only calibrated for a single origin point. The goal of characterization was to empirically measure the system's three-dimensional magnetic field response to input current values. The project also highlighted the limitations of the dipole model by revealing significant discrepancies between its predictions and the measured data, underscoring the need for a more accurate approach. This project aims to develop a machine-learning model based on the measured dataset to enable precise prediction and control of the system's 3D magnetic field.

2 Background

Electromagnetic actuation is an active research area in the biomedical field due to the promising application for minimally invasive surgery. The method has been demonstrated in previous studies to be useful on various clinical operations including micro-surgeries[6], catheter positioning [12, 8] and in vivo-capsule navigation for targeted drug delivery or sampling [13].

Magnetic actuation occurs when a magnetic element, i.e magnetic surgical tool tip, resides in an external field generated by a magnetic actuation system. For effective control of the applied forces and torques on the element, the interaction between the magnetic element and the external field is mathematically represented by Equations 1 and 2 which are simplifications of the Maxwell's equation [11].

$$F = \nabla(m \cdot B) \quad (1)$$

$$\tau = m \times B \quad (2)$$

Where F and τ are the force and torque on the magnetic element, m is the magnetic moment of the element and B is the magnetic field generated by the actuation system.

The above control representation indicates that the magnetic field and its gradient generated by the magnetic actuation system needs to be determined for effective control of the magnetic element. Previously, a dipole model has been considered but determining the exact magnetic moment of the core material was found to be a challenge and the result was less than satisfactory with Mean average percentage error reaching over 100% [2].

High inaccuracy of the dipole model and the fact that FEM modeling is computationally too intensive for real time control application signals the need for a new way of modeling our system. Since the actuation system has been experimentally characterized, this project focuses on utilizing the collected experimental data to model the magnetic actuation system with data driven modeling technique.

3 Literature Review

Three main modeling techniques have been implemented in the literature. The first one is analytical method such as biot-savart law [2, 11, 15] which describes electromagnets without iron core but performs inadequately in practical cases where iron cores are used. A simplified model is a dipole model where the core material is discretized into small cubes and represented as a collection of small components of dipole moments [9], however obtaining the number and type of parameters is not straightforward.

The Second method is using FEM modeling. FEM heavily relies on the fineness of discretization over its domain which increases its computational need rendering it unsuitable for real time control applications. Similarly to the analytical models, FEM models have limitation in dealing with non-linearity of the core material.

The other approach is using data driven machine learning models which have garnered a lot of interest in the recent years and have a promising characteristic to model any complex relationship for real time control application. The following section discusses the ideas behind Machine learning models and Artificial neural networks as it relates to magnetic actuation system modeling.

3.1 Machine learning models and Artificial neural networks (ANNs)

Machine learning and statistical algorithms are widely used to develop data-driven models. Among these techniques, deep neural networks stand out for their unique ability to approximate any continuous function [3], earning them the title of universal function approximators. Furthermore, deep neural

networks learn complex patterns and representations directly from data without requiring explicit prior knowledge about the underlying relationship. This capability makes them particularly suitable for modeling the relationship between input currents and the electromagnetic fields in a magnetic actuation system. By leveraging their ability to capture complex system behaviors, deep neural networks effectively address challenges such as the non-linear behavior of core materials under high magnetic fields and cross-magnetization effects between electromagnets [15].

Artificial Neural Networks are inspired by the inner workings of a biological brain and try to replicate its function [7, 1]. The simplest component of the network is a perceptron, analogous to a biological neuron. A perceptron receives signals from other neurons and produces a signal following the evaluation of an activation function of the weighted sum of input signals [4]. A collection of neurons as shown in Figure 2 form a network with the ability to model a desired system. Learning occurs when the network augments its weights to fit the training data using a gradient decent algorithm.

Currently, there is no clear method to find right model architecture and hyper-parameter values for a specific problem. The most common approach is to research what has been implemented for a similar problem and use that as a starting point in optimizing for ones problem. Smith et. al. lays out some techniques and recommendation the should be used for training a deep neural network [14].

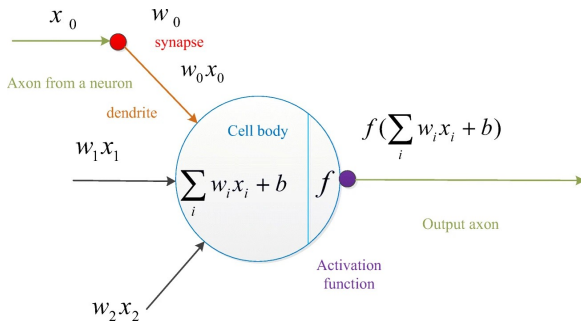


Figure 1: Perceptron [4]

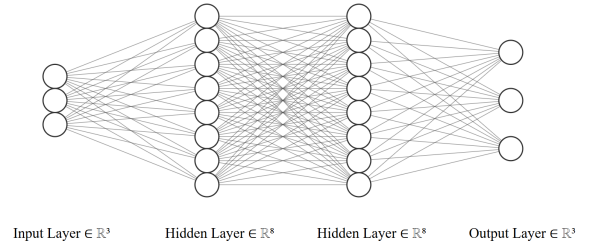


Figure 2: Fully connected ANN

Neural networks can be created in a specialized way for a specific application. Among the many types of networks the most prominent ones are feed forward network (FFN) or multilayer perceptrons, Convolutional neural networks (CNN), Recurrent neural networks (RNN) and Transformers. High level overview of each architecture is presented below.

Feed Forward networks (FFN)

Also known as Multi-layer Perceptions are the very simple form of artificial neural networks as shown in Figure 2. In this architecture, information moves in a single direction from input to output. These networks are used to process large amounts of data and are the foundation for many other neural network models.

Convolutional neural networks (CNNs)

Convolutional neural networks are designed to process data with a grid-like structure or spatial information. CNNs use convolutional layers to detect patterns like edges, textures, or shapes, which are then combined in a pooling layer to identify more complex features. Pooling layers reduce spatial dimensions, making the network efficient and less prone to over-fitting. They are heavily implemented in computer vision tasks, such as image classification, object detection, and image and video recognition.

Recurrent neural networks (RNNs)

Recurrent Neural Networks (RNNs) are a class of neural networks specifically designed to process sequential data and capture temporal or sequential dependencies. Unlike feedforward networks, RNNs have connections that form loops, allowing information to persist as it flows through the network. This architecture enables RNNs to maintain a "memory" of previous inputs, making them suitable for tasks where the order of data matters, such as natural language processing, time-series forecasting, and speech recognition. At each step, the hidden state of the RNN is updated based on the current input and the hidden state from the previous step, enabling it to learn patterns over sequences. However, standard

RNNs face challenges such as vanishing or exploding gradients when dealing with long-term dependencies. To address this, advanced architectures like Long Short-Term Memory (LSTM) networks and Gated Recurrent Units (GRUs) were developed, incorporating mechanisms to selectively remember or forget information over long sequences.

Transformers

Transformers, unlike RNNs, process entire sequences in parallel rather than sequentially, thanks to their self-attention mechanism. This allows them to capture long-range dependencies more efficiently and handle complex relationships within the data without the bottlenecks associated with sequential processing. Initially developed for natural language processing tasks like translation and text generation, transformers have demonstrated remarkable versatility across various domains. In computer vision, they analyze images by dividing them into sequences of patches, enabling efficient modeling of spatial relationships. Similarly, in time-series forecasting, transformers excel at capturing intricate temporal patterns and trends over long durations, making them a powerful tool for sequence-based data across disciplines.

3.2 Embedding Physics into Neural Networks

There are three different methods to embed physics in machine learning models. The first one is introduced by Raissi et al. [10] as physics informed neural networks (PINNs) which consists of imposing mathematical constraint described by general non-linear partial differential equation on neural networks prediction. The constraint restrains the prediction to abide by the fundamental laws of physics. The physics constraints are imposed by introducing an additional loss function of partial differential equation on the conventional neural network. Figure 3 illustrates how to impose a viscous Burgers' equation on the neural network output. The network is trained by minimizing the total loss via gradient based optimizers, such as Adam and L-BFGS, until the loss is smaller than a threshold value ϵ .

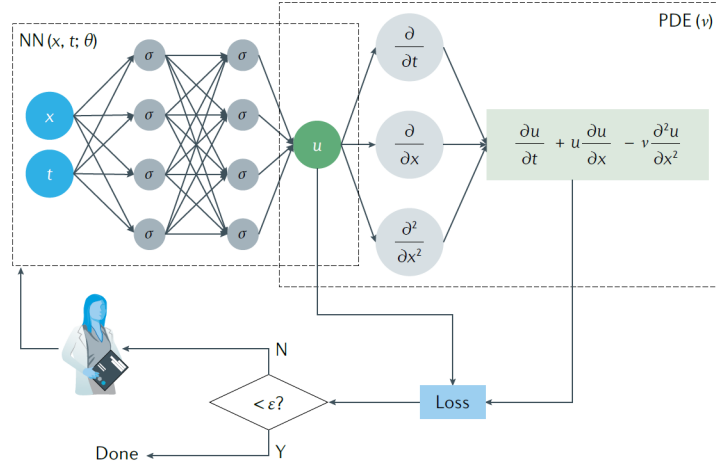


Figure 3: PINNs and learning bias for viscous Burgers' equation [5]

Another approach to ensuring that predictions align with physical laws is through the design of machine learning model architectures. A notable example is the convolutional neural network (CNN) architecture, which inherently respects invariance under a group of symmetries [5]. While this principled method offers a robust framework for incorporating physical constraints, the development of new architectures tailored to specific problems remains a significant challenge. This limitation restricts the applicability of this approach to tasks characterized by relatively simple and well-defined physical principles.

The most common approach of embedding physics in neural networks is implicitly through the training dataset. Measurement data reflect the underlying physical principles that dictate their generation, and, in principle, can be used as a weak mechanism for embedding these principles into an ML model during its training phase. However, this approach depends heavily on the availability and quality of observational data.

3.3 Previous Studies on Electromagnetic Field Prediction

A study by Sikorski et al. performed FEM modeling of 6 mobile electromagnetic coils capable of generating 40mT. Due to the natural inaccuracies of the FEM model, the authors used measurement based calibration by introducing an exponential term that accounts for the distance from the coil to the prediction point. The validation test showed that the model has the mean percent error of 3.36% for a single coil and 15.21% for the entire system (6 superimposed coils) inside a 10cm diameter spherical workspace [12].

Herris et al. highlighted that their finite element method (FEM) simulation exhibited an error of up to 17%, primarily due to challenges in accurately representing the electromagnetic coil. Factors such as manufacturing induced gaps in the coil winding and variations in core material properties, along with the inherent approximation errors of the FEM method, contributed to this discrepancy. To address these limitations, the authors opted for an artificial neural network (ANN) approach, incorporating a simulated annealing method for initializing the network's weights. The ANN architecture consisted of a single hidden layer with seven neurons for each magnetic field axis. This approach achieved a goodness-of-fit R^2 value of 0.9871 and a root mean square error (RMSE) of 0.0153 (no unit was specified) [6].

Yu et al. inspired to model the non-linear magnetization region by employing random forest and artificial neural networks. The ANN model has an input size of 11 that include input current and position vector and output size of three that consists of the x, y and z component of the predicted magnetic field. The model has three hidden layers with 100, 50 and 25 neurons, respectively and a hyperbolic tangent is used as an activation function. The ANN model achieved a RMSE of 3.01mT and goodness of fit R^2 value of 0.99 over the measured data [16].

Zhao et.al showed how to use a transformers-based PINNs model to predict the solution of high dimensional PDEs such as the Navier-Stokes equation with very high accuracy and proposed a wavelet activation function shown in Equation 3 [17]. ω_1 and ω_2 are learned variables in the wavelet equation as the motivation of using this activation function is to universally approximate any suitable activation profile using sine and cosine functions drawn from the intuition of Fourier transform. The authors propose to extend the wavelet function to the general deep-learning tasks.

$$\text{wavelet}(x) = \omega_1 \sin(x) + \omega_2 \cos(x) \quad (3)$$

4 Objectives

The main objective of this project is to develop a data-driven model for an 8-coil magnetic actuation system using an empirical dataset.

4.1 Specific Objective

Specific objectives of the project include:

- Benchmark the developed performance with the literature
- Compare the performance of the developed model with the dipole model.

5 Approach

As discussed in the literature review, neural networks are a powerful tool for modeling complex systems and are recognized as universal function approximators, offering advantages over other statistical modeling methods. For the current task, the most suitable neural network architectures are Feedforward Networks (FFNs) and Convolutional Neural Networks (CNNs). CNNs, however, predict values at grid points and require the computation of gradients through interpolation functions, which can introduce inaccuracies. In contrast, FFNs can predict magnetic fields at arbitrary input locations, with gradients being directly calculated. Therefore, an FFN architecture with current and position as input variables is selected for this task.

The physics of electrodynamics will be embedded into the network by the use of the training data. In order to do that, a significant amount of direct field measurement data for accuracy is required for training. Previous data measurement collected as part of system characterization in ref [2] is used as part of the training dataset. It includes 112,832 data point measurement for 112 distinct input current

vectors. This dataset contains single coil magnetization at 2 amps increment and at 1,008 equidistant grid point locations in the workspace.

Additional field measurements with randomized input current vector has been conducted as part of this project. The data collection processes is the same as [2] but using a new feature added in Qt GUI application as shown in 9 in appendix A. The new feature allows to specify the number of randomized current values and have the flexibility to have measurement taken at grid point by checking the checkbox, otherwise data collection will be at a randomized location. The new Randomized dataset contained 40,416 measurements data points at grid point and 25,216 data points at random location within the workspace.

In total, 178,464 data points were prepared for training our model. The dataset breakdown is shown in Table 1 and the dataset distribution in the workspace is visualized in Figure 10 in the Appendix. As show in the figure, the total 3D space were covered with the training datasets and higher number of data is contained at grid point locations indicated by the peaks in Kernel Density Estimate plots shown in the diagonal distribution plots. Similarly the current distribution plot is shown in Figure 11 in the Appendix. The dataset was divided in to 72.25% for training, 12.75 for validation and the rest 15% for testing.

	<i>Type of measurement</i>	<i>No. of data pts</i>	<i>Distinct input current values</i>
1	Single coil magnetization	112,832	112
2	Randomized input current on grid pt in space	40,416	54
3	Randomized input current at random location	25,216	311
	Total	178,464	477

Table 1: Training dataset breakdown

To Find the optimal model architecture (number of hidden and nodes per layer) and hyper-parameters for our specific case custom tuning was implemented. For this purpose, a software tool called Optuna was used to optimize parameters, along with a tool called MLflow for tracking and logging the experiments. The main hyper-parameters considered for optimization were activation function, batch size and learning rate using Adam optimizer. The optimization was conducted within the PyTorch framework, with the objective of minimizing the validation root mean square error (RMSE). A total of 330 experimental runs were performed to determine the best model architecture and hyper-parameters. The results are presented in the following section.

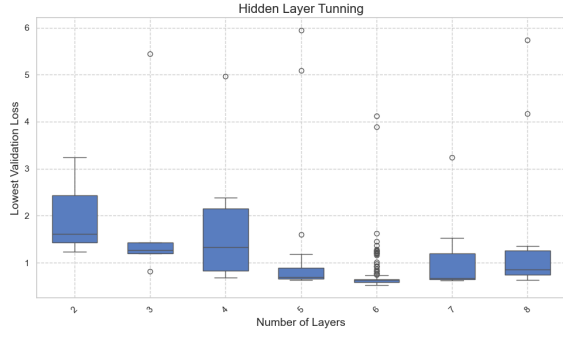
Model optimization were performed in two stages. On the first stage 250 runs were conducted considered Number of layer (2 - 8), Hidden layer nodes per layers (2 - 250), activation function (wavelet, tanh, ReLU), Batch size (32, 64, 96, 128), learning rate (1e-5, 1e-3). Based on the first stage results, 80 runs were conducted to fine tune the model by only considering the following critical parameters Hidden layer nodes per layers (250 - 500), activation function (Leaky-ReLU, ReLU), batch size (128, 192) and learning rate (0.0004, 0.001). The result for both stages were merged and presented in the following section.

6 Result

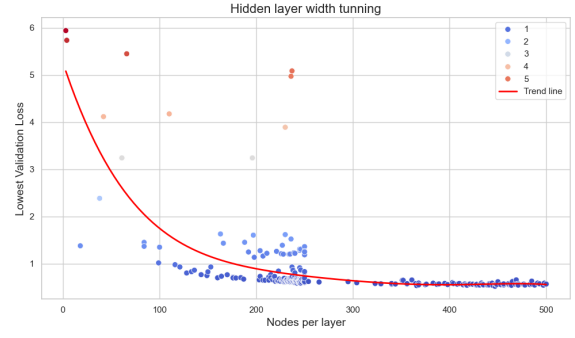
6.1 Model architecture Tuning

The model architecture tuning involved determining the optimal number of hidden layers and nodes per layer. The results from our optimization are plotted in Figures 4a and 4b. Figure 4a shows that as the number of hidden layers increases, the median validation loss of the model decreases up to 6 layers. Beyond that, higher losses are observed. Additionally, six hidden layers resulted in a small interquartile range, indicating that it consistently produces a high-performing model across varying parameters. Therefore, six hidden layers is identified as the optimal configuration.

For the optimal number of nodes per hidden layer, the results are presented as a scatter plot in Figure 4b. The plot shows scattered results for node counts up to 250, with a consistent decrease in validation loss beyond that. The trend line suggests that increasing the number of nodes per layer generally lowers the validation loss, but with diminishing returns as the number of nodes increases.



(a) Hidden layer tuning



(b) Node per layer tuning

Figure 4: Model architecture tuning

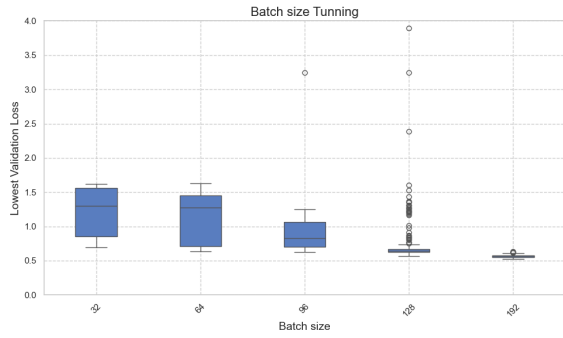
6.2 Hyper-parameter tuning

Hyperparameter tuning was performed on batch size, learning rate, and activation function.

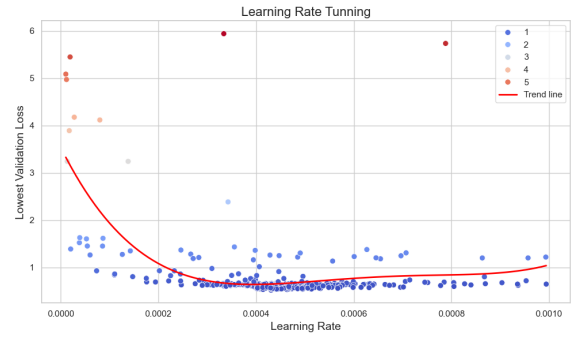
For batch size, it was observed that increasing the number of data points in a training batch led to a lower validation loss, as shown in Figure 5a. The optimal batch size was determined to be 192, as it produced the lowest validation loss.

The scatter plot of validation loss against learning rate, shown in Figure 5b, reveals a sharp decrease in validation loss as the learning rate increases to 0.0004, after which the loss gradually increases. The optimal learning rate was found to be 0.00043.

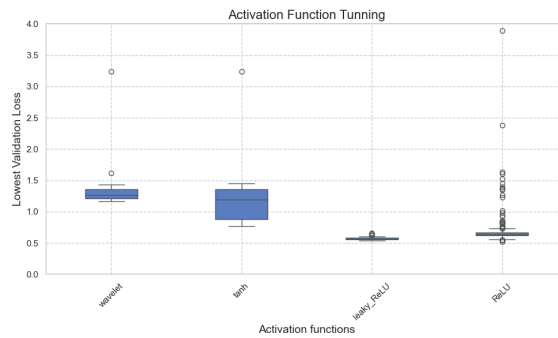
The activation function was another key hyperparameter considered for tuning. As shown in Figure 5c, the wavelet and tanh functions were less optimal. In contrast, Leaky-ReLU consistently achieved the lowest validation loss within a compact range of values. ReLU, while exhibiting a slightly wider range of validation losses, had some outliers performing better and others worse than the other activation functions, yet still achieved low median loss values. Given that ReLU produced some of the best-performing outliers, and considering the introduction of Leaky-ReLU in the second stage of optimization, ReLU was chosen as the best activation function.



(a) Batch size tuning



(b) Learning rate tuning



(c) Activation function tuning

Figure 5: Hyper-parameter tuning

6.3 Optimized model

Based on the optimization result, the best model architecture has 6 hidden layers with 477 nodes per layer. The optimal hyper-parameters used for training were a batch size of 192, learning rate of 0.00043 with Adam optimizer, and activation function of ReLU function. The model information is summarized in Table 2 and the test metrics for our best model is bench-marked against previous studies in Figure 6 as a heat map plot. The figure visualizes the performance metrics comparison of Mean Absolute Percentage Error (MAPE), Root Mean Square Error (RMSE), and Mean Absolute Error (MAE) across different studies in the literature against our optimized model’s test results.

Our model achieves a MAPE of 3.94%, which is significantly lower than the values reported by Sikorski et al. (15.21%) and Heris et al. (17.00%). This indicates that our model demonstrates a much better prediction accuracy relative to the ground truth in percentage terms. Our model achieves an RMSE of 0.71 mT, which is better than Yu et al. (3.01 mT) and comparable to Charreyron et al. (0.83 mT). Our model achieves an MAE of 0.38 mT, which is lower than Charreyron et al. (0.51 mT). MAE directly quantifies the average error, showcasing our model’s superior reliability in error minimization. This comparison benchmarks our model to be the best in the literature and suggests that our parameter optimizations have successfully enhanced performance relative to prior studies.

We also evaluated the performance of our model by conducting a speed test using a system with NVIDIA GeForce RTX 4090 GPU with 24 GB VRAM using CUDA 11.7. The model achieved a inference time of approximately 0.001 seconds for 100 predictions, demonstrating its suitability for real-time control applications.

Model architecture	
Hidden layers	6
Nodes per layer	447
Parameter	
Learning rate	
Activation function	ReLU
Batch size	192
Test Metrics	
RMSE	0.71
MAE	0.38
MAPE	3.94%
pitch angle	2.18
yaw angle	3.69

Table 2: Optimized model parameter and performance

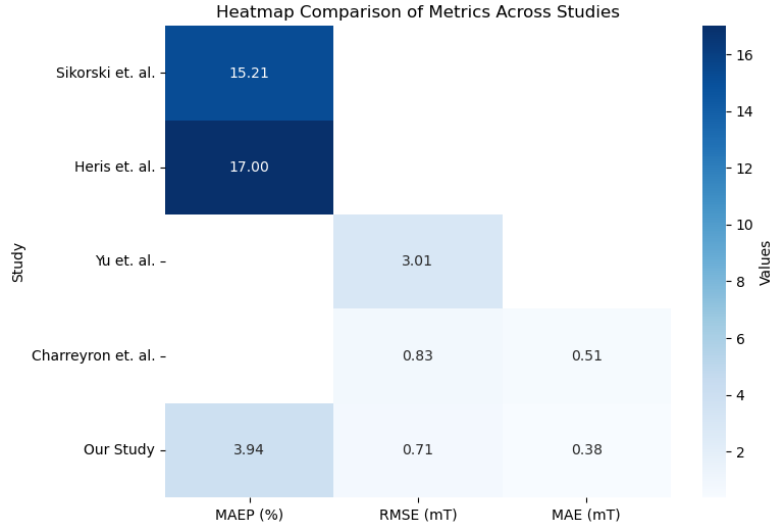


Figure 6: Benchmarking our best model with studies in the literature

6.4 Model prediction vs Measurement

Next, we compared the model prediction with three test cases of direct field measurement data at grid point location. The test cases are have been used to test super positioning and the dipole model in the previous report [2] and were not part of the training data set nor where used for validation or testing. Magnetic field vectors have predicted for grid point locations where measurement have been taken. The input current vectors for the three test cases are as follows:

- Test 1: [5, 5, 5, 5, 5, 0, 5, 5]
- Test 2: [10, 0, -10, -5, 5, 0, 0, 10]
- Test 3: [-10, -5, -10, 0, 0, 0, 5, 10]

Figure 3 shows a visual comparison of magnetic fields between our model prediction and measured data for test 1. Comparison plot for test 2 and 3 is given in the Appendix D. The plots shows magnetic field as a quiver plot at each measurement grid point. To distinguish between model prediction and measured data different color schemes were used. However, the two plots look almost identical with minor differences.

Mean absolute percentage error (MAPE) and mean absolute error (MAE) of our model for each test case is tabulated in Table 3. The performance metrics are evaluated across the three Cartesian components (x, y, z), the vector norm, and the angular parameters pitch and yaw angle (ϕ and θ respectively). The x component exhibits relatively lower MAE values across all tests, with slight variations. However, MAPE values are highly variable, reaching a peak of 173.9% in Test 3. The y component shows a significant MAPE increase in Test 1 (457.6%) and Test 2 (343.4%), indicating substantial relative error despite moderate MAE values. The z component maintains consistent MAE values across tests, with slightly higher errors compared to x and y but lower MAPE values overall.

The norm showed MAE values ranging from 1.23 mT to 1.43 mT and maximum MAPE of 17.3% that is equivalent to FEM modeling error calculated by Heris et al. [6]. The Lower accuracy observed on this test case, compared to the test result, may be due to the presence of many repeated input current values in the dataset used for training, validation, and testing. Specifically, there are 477 distinct current values in 178,464 data points, meaning there is a high likelihood that most of the input current values used for training have been encountered during testing leading to a very accurate test result. This finding, however, suggests that incorporating a wider variety of current values could further improve the model.

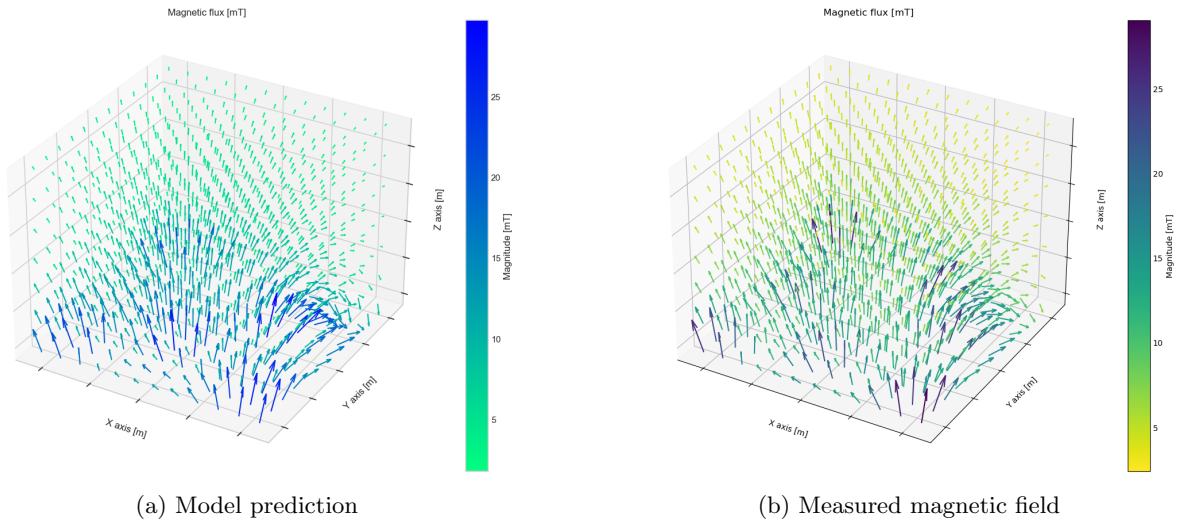


Figure 7: Predicted vs measured magnetic field for test 1.

Input command current = [5, 5, 5, 5, 5, 0, 5, 5] for coil 1 to 8 consecutively

	Test 1		Test 2		Test 3	
	MAE [mT]	MAPE (%)	MAE [mT]	MAPE (%)	MAE [mT]	MAPE (%)
X	0.70	75.0	1.13	38.3	0.91	173.9
Y	0.80	457.6	1.99	343.4	0.98	47.4
Z	1.59	49.6	1.21	64.5	1.24	113.4
Norm	1.43	17.3	1.27	11.0	1.23	9.7
ϕ (°)	6.24		5.74		5.97	
θ (°)	9.58		10.47		8.10	

Table 3: Mean absolute and mean absolute percentage error between the superimposed and measured magnetic fields

6.5 Performance comparison between FFN model, dipole model and super-positioned data

In this section we compare our optimal FFN model with dipole model and super-positioned data from the preceding report [2]. The comparison is for the same three test cases of direct measurement data as in the previous section. The comparison is plotted in Figure 8. The plot shows super-positioning data in blue, Dipole model performance in orange and our models performance in green. Mean absolute error is shown in the left axis for the bar chart and right axis shows mean absolute percentage error

and the corresponding values are represented by a star with light color. As shown in the figure, the dipole model performs worse in all cases and for both test metrics. Our model performs slightly worse than the super-positioned data except for test 3 in the z-direction where our model outperforms the super-positioned data. Overall, this comparative plot indicates our model is performing close to a super positioned data which indicates the reliability of our model output.

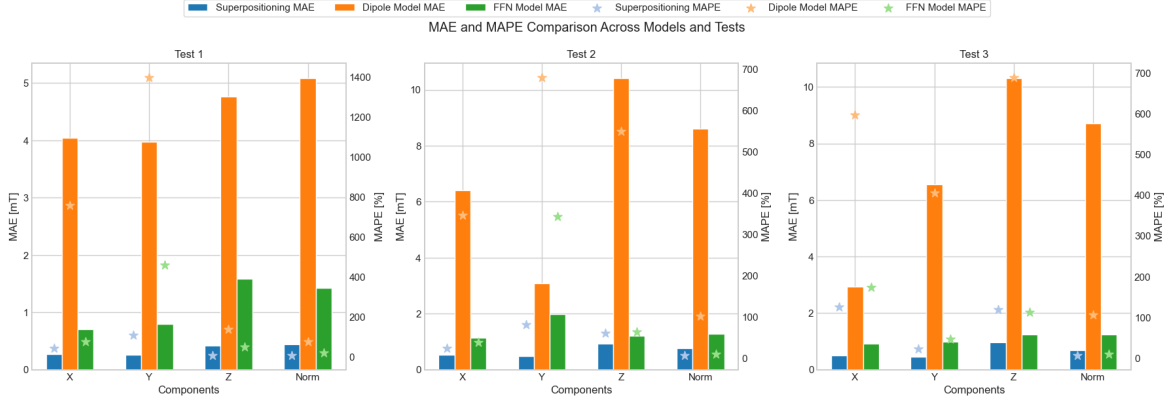


Figure 8: Performance comparison between FFN model, dipole model and superpositioned data

6.6 Measured vs command current

Up to this point, Command current has been used as input current. Our optimized model implicitly models the mapping between the command current and the actual current supplied to the coils. The actual mapping is accomplished by a current controller in the system that ensured the set command current is delivered to the coils. As part of the data collection, the current supplied to the coils were also measured and collected. to observe the relationship between the command and measured (supplied) current, The command vs measured current of our dataset is plotted for each coil in Figure 14 in the Appendix E. The plot shows that the command vs measured current does not correlated for all but coil 1, 2 and 6. This finding was unexpected and the cause of this non correlation should be further investigated.

In the previous report [2], it was noted that energizing one coil causes a current to be observed in neighboring coils, even when their command current is set to zero. This might be due to fluctuation of the actual coil supplied current with time (by the inability of the current controller to supply steady current values) causing cross-magnetization or interference with the signal wires. This also may lead to the current controller of the neighboring coils to counteract the cross-magnetization effect. Alternatively, a faulty current controller could also be responsible. These and other potential causes should be thoroughly investigated as part of a future work.

To isolate the current controller effect, a new model was trained using the same parameter as our optimized model but with measured current in the training dataset. Due to the randomness of of the input current, the measured current model performed less accurately to our dataset. The test metrics calculated are 0.86mT RMSE, 0.63mT MAE, and 11.8% MAPE. When tested for the three test cases, the measured current model showed upwards of 40% MAPE. This indicates either the model is not fitting the data well so it needs to be optimized for the this new dataset or the measured current value is too random to be modeled.

7 Conclusion and Recommendation

Building upon the work presented in [2], this study focuses on developing a data-driven model for an 8-coil magnetic actuation system. In addition to a single-coil magnetization dataset, new direct field measurements were conducted at randomized current and spatial locations within the workspace. A total of 178,464 data points were collected for model training, validation, and testing. Subsequently, a feed-forward neural network model was optimized, and its test results were benchmarked against existing models in the literature. The results demonstrate that our model outperforms previous models while

offering the added capability of real-time control. However, test cases suggest that the model could further benefit from additional training with more diverse and randomized current values.

Further analysis of the our dataset revealed an unexpected lack of correlation between the commanded current and the actual current supplied to the coils, appearing instead to be random. Potential causes, such as cross-magnetization, field interference with signal wires, or a malfunctioning current controller, have been hypothesized based on preliminary observations. However, further investigation is required to confirm and accurately identify the root cause.

References

- [1] H.-D. Block. The perceptron: A model for brain functioning. i. *Reviews of Modern Physics*, 34(1):123, 1962.
- [2] G. M. Girma. Magnetic actuation system dataset generation and characterization.
- [3] K. Hornik, M. Stinchcombe, and H. White. Multilayer feedforward networks are universal approximators. *Neural networks*, 2(5):359–366, 1989.
- [4] R. Hou, D. Zhou, R. Nie, D. Liu, and X. Ruan. Brain ct and mri medical image fusion using convolutional neural networks and a dual-channel spiking cortical model. *Medical & biological engineering & computing*, 57:887–900, 2019.
- [5] G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, and L. Yang. Physics-informed machine learning. *Nature Reviews Physics*, 3(6):422–440, 2021.
- [6] P. Kazemzadeh Heris and M. B. Khamesee. Design and fabrication of a magnetic actuator for torque and force control estimated by the ann/sa algorithm. *Micromachines*, 13(2):327, 2022.
- [7] A. Krogh. What are artificial neural networks? *Nature biotechnology*, 26(2):195–197, 2008.
- [8] V. N. Le, N. H. Nguyen, K. Alameh, R. Weerasooriya, and P. Pratten. Accurate modeling and positioning of a magnetically controlled catheter tip. *Medical physics*, 43(2):650–663, 2016.
- [9] V. T. Le. Accurate modelling and positioning of a magnetically-controlled catheter tip. 2015.
- [10] M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational physics*, 378:686–707, 2019.
- [11] A. Schonewille. Maximizing workspace accessibility in magnetic actuation of tethered microsurgical tools using non-uniform magnetic fields, 2022.
- [12] J. Sikorski, I. Dawson, A. Denasi, E. E. Hekman, and S. Misra. Introducing bigmag—a novel system for 3d magnetic actuation of flexible surgical manipulators. In *2017 IEEE International Conference on Robotics and Automation (ICRA)*, pages 3594–3599. IEEE, 2017.
- [13] M. Sitti, H. Ceylan, W. Hu, J. Giltinan, M. Turan, S. Yim, and E. Diller. Biomedical applications of untethered mobile milli/microrobots. *Proceedings of the IEEE*, 103(2):205–224, 2015.
- [14] L. N. Smith. A disciplined approach to neural network hyper-parameters: Part 1—learning rate, batch size, momentum, and weight decay. *arXiv preprint arXiv:1803.09820*, 2018.
- [15] L. Yang, J. Jiang, F. Ji, Y. Li, K.-L. Yung, A. Ferreira, and L. Zhang. Machine learning for micro-and nanorobots. *Nature Machine Intelligence*, 6(6):605–618, 2024.
- [16] R. Yu, S. L. Charreyron, Q. Boehler, C. Weibel, C. Chautems, C. C. Poon, and B. J. Nelson. Modeling electromagnetic navigation systems for medical applications using random forests and artificial neural networks. In *2020 IEEE International Conference on Robotics and Automation (ICRA)*, pages 9251–9256. IEEE, 2020.
- [17] Z. Zhao, X. Ding, and B. A. Prakash. Pinnsformer: A transformer-based framework for physics-informed neural networks. *arXiv preprint arXiv:2307.11833*, 2023.

Appendix

A GUI for Randomized Dataset Collection

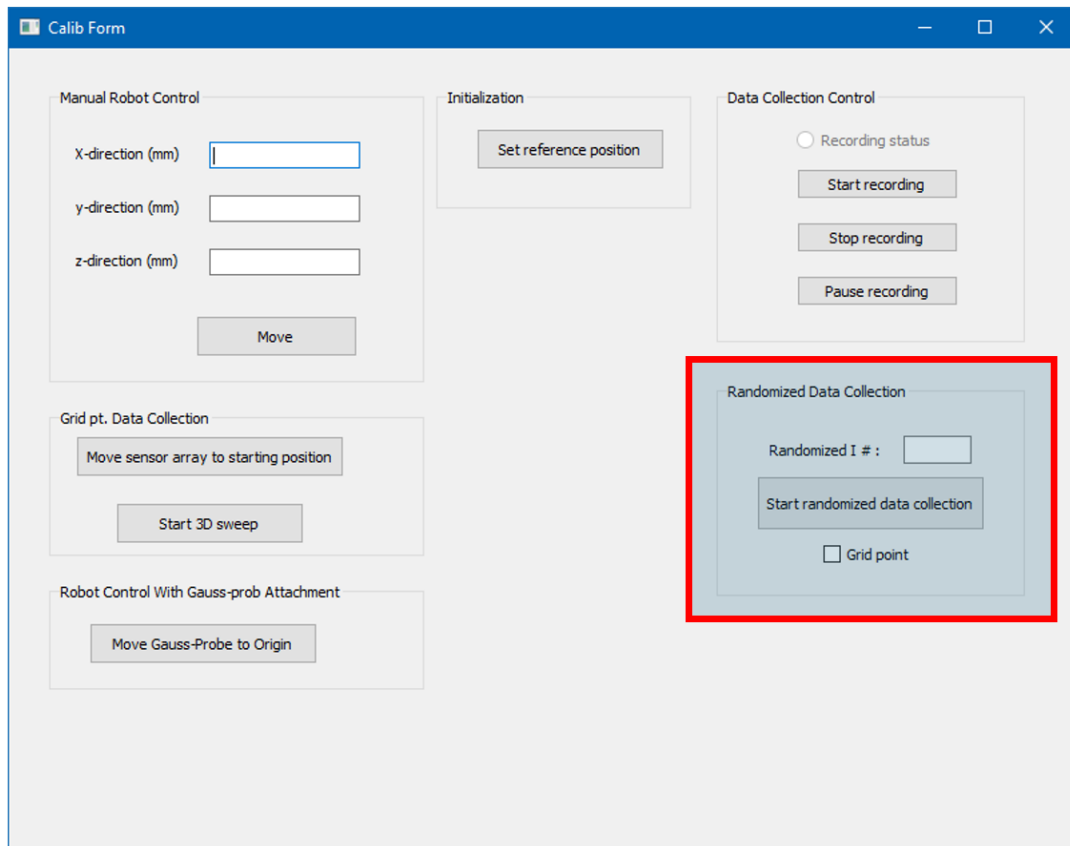


Figure 9: New feature, shown in red, for randomized data collection

B Training Data Distribution In The Workspace

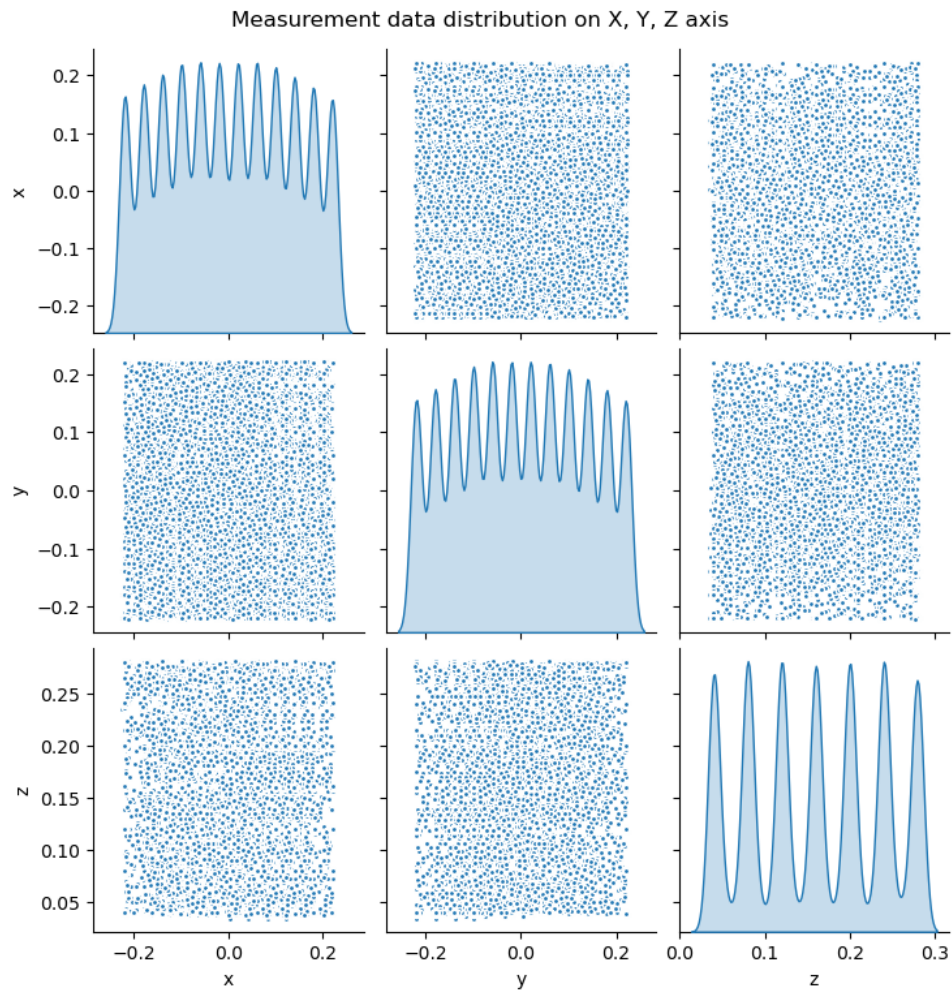


Figure 10: Training data distribution on x, y and z axis

C Input Current Vector Distribution of the Training Dataset

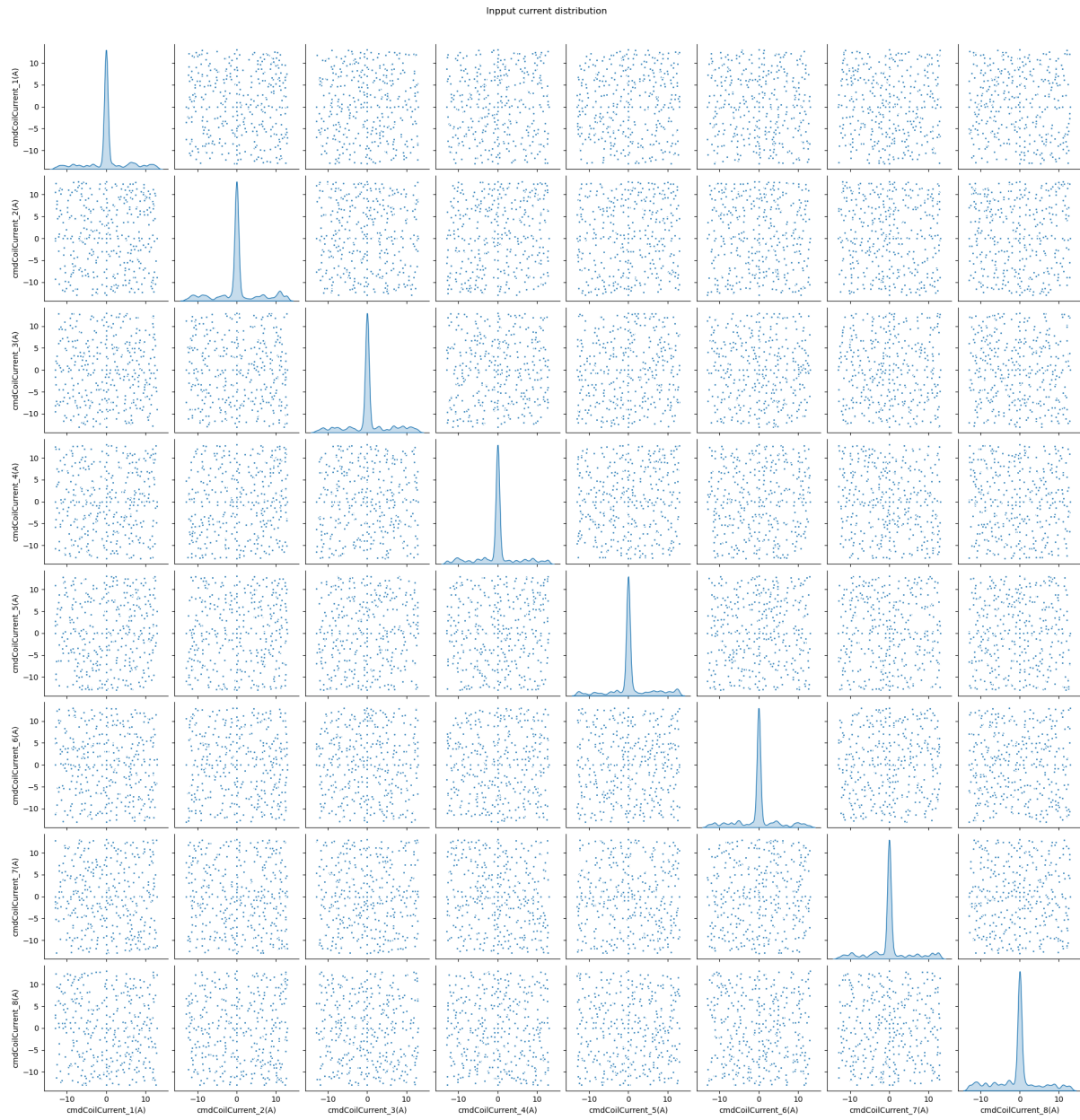


Figure 11: Input current vector distribution of the training dataset

D FFN Prediction vs Measured Data

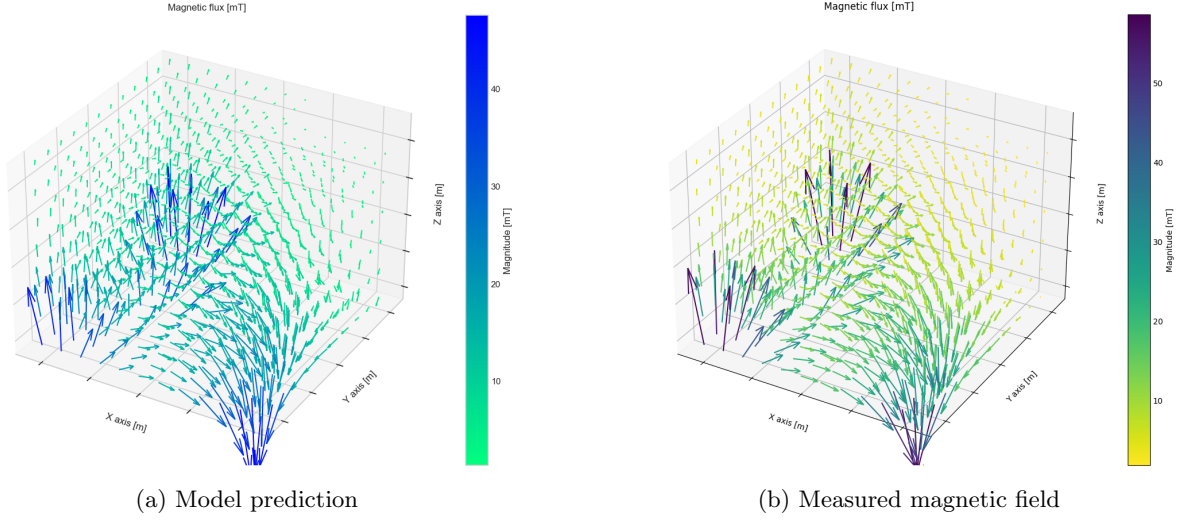


Figure 12: Predicted vs measured magnetic field for test 2.
Input command current = $[10, 0, -10, -5, 5, 0, 0, 10]$ for coil 1 to 8 consecutively

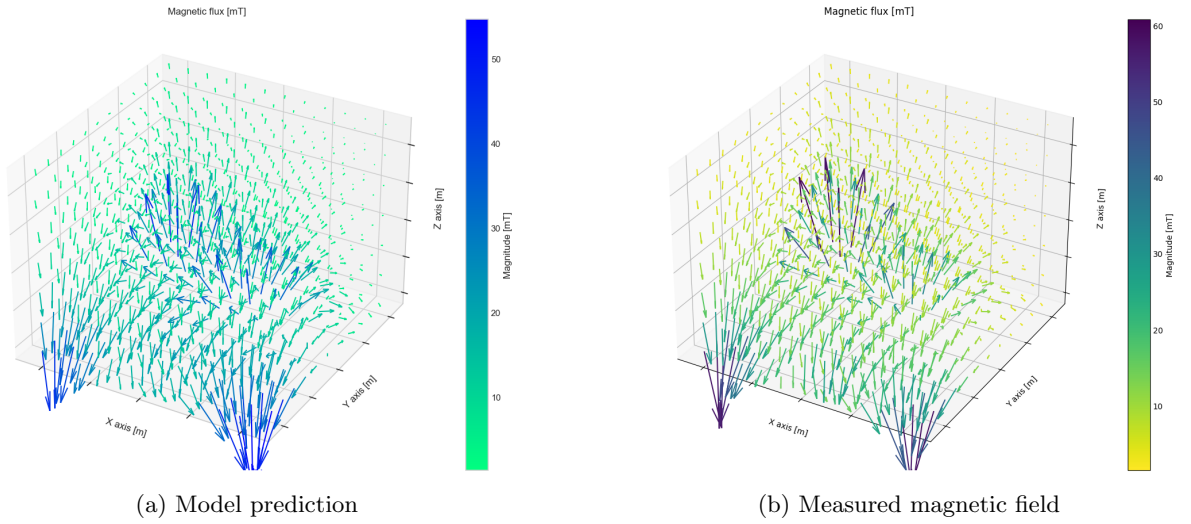


Figure 13: Predicted vs measured magnetic field for test 3.
Input command current = $[-10, -5, -10, 0, 0, 0, 5, 10]$ for coil 1 to 8 consecutively

E Command vs Measured Current of the Training Dataset

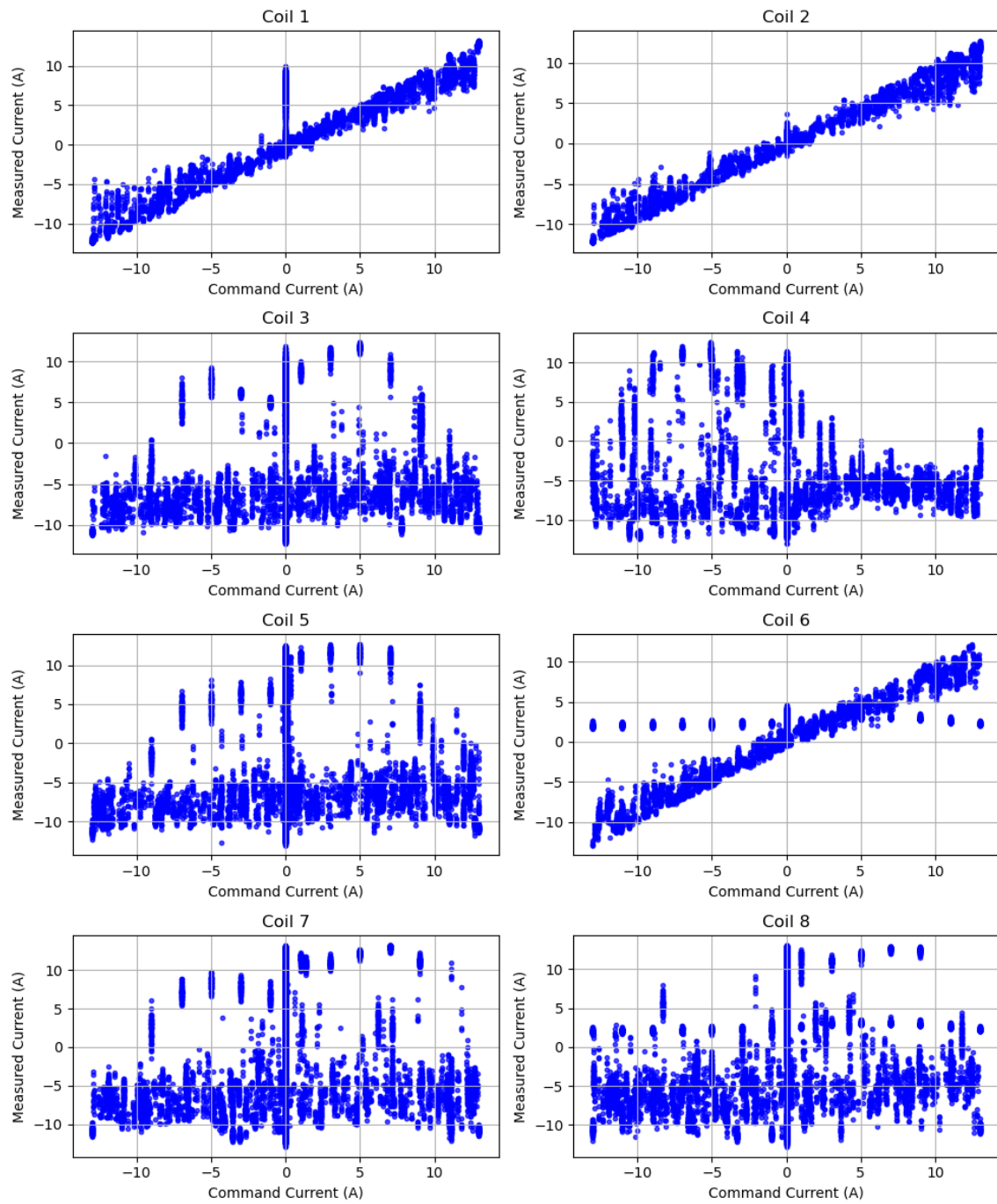


Figure 14: Command vs measured current of the training dataset